Advanced Topics in Numerical Mathematics

Final Project: Model Solving with Neural ODE

Nikoloz Turashvili

KIU

January 29, 2026

The Cucker–Smale Flocking Model

Model Description and Mathematical Structure

- Collective motion and flocking behavior
- Continuous-time interacting particle system
- Velocity alignment through nonlocal interactions

Motivation and Background

The Cucker–Smale model was introduced to explain how consensus and flocking can emerge in multi-agent systems without any central coordination. It provides a mathematical framework to study how, under suitable initial conditions, a group of agents (e.g., birds or fish) converges to a common velocity through simple distance-dependent local interactions. The model is remarkable in that it establishes convergence guarantees based only on the initial state, and demonstrates how coherent collective behavior can emerge from simple interaction rules.

Modelling Collective Behavior: Flocking

"Flocking" behavior is a particular kind of collective behavior that can be easily found in nature while observing the collective motion of a large number of individuals.





The Cucker–Smale Model with Communication Rates

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i, \qquad \frac{d\mathbf{v}_i}{dt} = \frac{\lambda}{N}\sum_{j=1}^{N} \psi_{ij}(t)\big(\mathbf{v}_j - \mathbf{v}_i\big).$$

- λ measures the strength of the interaction force between individuals.
- The function $t \mapsto (\psi_{ij}(t))_{i,j}$ is called the *communication rate*.
- $\psi_{ij}(t) \geq 0$ characterises the influence of individual j on individual i.

A classical choice is

$$\psi_{ij}(t) = \psi\big(\|\mathbf{x}_i(t) - \mathbf{x}_j(t)\|\big),$$

where ψ is usually chosen positive and decreasing, for example

$$\psi(r) = \frac{1}{(1+r^2)^{\beta}},$$

for a given $\beta \geq 0$.

The Cucker–Smale Flocking Model

We consider a system of N interacting agents moving in $\mathbb{R}^d$. Each agent is characterized by:

- Position: $x_i(t) \in \mathbb{R}^d$
- Velocity: $v_i(t) \in \mathbb{R}^d$

The evolution of the system is governed by the following system of ordinary differential equations:

$$\dot{x}_i(t) = v_i(t)$$

$$\dot{v}_i(t) = \frac{1}{N} \sum_{j=1}^{N} \psi\big(\|x_j(t) - x_i(t)\|\big)\big(v_j(t) - v_i(t)\big)$$

where the interaction kernel is typically chosen as:

$$\psi(r) = \frac{1}{(1+r^2)^{\beta}}, \quad \beta \geq 0.$$

Variables and Parameters

- N: number of agents
- d: spatial dimension
- $\mathbf{x}_i(t) \in \mathbb{R}^d$: position of agent i at time t
- $\mathbf{v}_i(t) \in \mathbb{R}^d$: velocity of agent i at time t
- $\lambda > 0$: interaction strength parameter (scales the alignment force)
- $\psi_{ij}(t) \geq 0$: communication rate measuring the influence of agent j on agent i
- $\psi(r)$: distance-dependent interaction kernel, decreasing with distance
- β: decay parameter controlling how fast interactions weaken with distance

Physical Interpretation

- The first equation is purely kinematic: positions evolve according to velocities.
- The second equation describes velocity alignment.
- Each agent adjusts its velocity toward the velocities of the others.

Classification of the Cucker–Smale Model

Classification by Order

The system involves only first-order time derivatives:

$$\frac{d\mathbf{x}_i}{dt}, \qquad \frac{d\mathbf{v}_i}{dt}.$$

The highest derivative appearing in the system is of first order. Therefore, the Cucker–Smale model is a *first-order ODE system*.

Classification by Type and Linearity

- The model is a system of ODEs (not a single equation).
- It is coupled, since each agent interacts with all others.
- It is nonlinear, because the interaction weights depend nonlinearly on the agents' positions:

$$\psi_{ij}(t) = \psi\big(\|\mathbf{x}_i(t) - \mathbf{x}_j(t)\|\big), \qquad \psi(r) = \frac{1}{(1+r^2)^{\beta}}.$$

The presence of the nonlinear function $\psi(\cdot)$ makes the right-hand side nonlinear in the state variables.

Continuity, Uniqueness

and Practical Applications of the Cucker–Smale Model

- Continuity of the Cucker–Smale vector field
- Uniqueness of solutions
- Practical applications of the Cucker–Smale model

Continuity of the Cucker–Smale Vector Field

We consider the Cucker–Smale system

$$\dot{y}(t) = f(y(t)), \qquad y = (x_1, \ldots, x_N, v_1, \ldots, v_N) \in \mathbb{R}^{2Nd}.$$

For $i = 1, \ldots, N$,

$$f_{x_i}(y) = v_i, \qquad f_{v_i}(y) = \frac{\lambda}{N} \sum_{j=1}^{N} \psi(\|x_i - x_j\|)(v_j - v_i),$$

where

$$\psi(r) = (1 + r^2)^{-\beta}.$$

Claim. The vector field

$$f : \mathbb{R}^{2Nd} \to \mathbb{R}^{2Nd}$$

is continuous.

Proof of Continuity

Let $y^{(n)} \to y$ in $\mathbb{R}^{2Nd}$.

Position components.

$$f_{x_i}(y^{(n)}) = v_i^{(n)} \to v_i = f_{x_i}(y).$$

Velocity components. Since

$$x_i^{(n)} \to x_i, \qquad v_i^{(n)} \to v_i,$$

and since the norm and ψ are continuous,

$$\psi(\|x_i^{(n)} - x_j^{(n)}\|)(v_j^{(n)} - v_i^{(n)}) \to \psi(\|x_i - x_j\|)(v_j - v_i).$$

As the sum is finite,

$$f_{v_i}(y^{(n)}) \to f_{v_i}(y).$$

Hence $f(y^{(n)}) \to f(y)$, and f is continuous.

Existence of Solution (Peano's Theorem)

We have shown that the vector field $f(y)$ of the Cucker–Smale system is continuous with respect to y.
We consider the initial value problem (IVP):

$$\frac{d}{dt}y(t) = f(y(t)), \quad y(t_0) = y_0,$$

where

$$y(t) = (x_1(t), \ldots, x_N(t), v_1(t), \ldots, v_N(t)) \in \mathbb{R}^{2Nd},$$

and $f : \mathbb{R}^{2Nd} \to \mathbb{R}^{2Nd}$ is defined by

$$\begin{cases} \dot{x}_i = v_i, \\ \dot{v}_i = \dfrac{\lambda}{N} \sum_{j=1}^{N} \psi(\|x_i - x_j\|)(v_j - v_i). \end{cases}$$

Since $f(y)$ is continuous, by **Peano's Existence Theorem**, there exists at least one solution $y(t)$ of the IVP on some time interval

$$t \in [t_0 - \alpha,\ t_0 + \alpha], \quad \alpha > 0.$$

Practical Applications of the Cucker–Smale Model

The Cucker–Smale flocking model is not only of theoretical interest, but also has important applications in engineering and control of multi-agent systems.

- **Multi-agent coordination and swarm control:** The model is widely used to design flocking and consensus protocols for multi-agent systems under realistic conditions such as communication delays, packet losses, and network attacks (e.g., DoS attacks).
- **UAV formation control and robotics:** Variants of the Cucker–Smale model are applied to control formations of unmanned aerial vehicles (UAVs) and mobile robots, enabling alignment, velocity consensus, and collision avoidance using only local interaction rules.
- **Decentralized and distributed control:** The model provides a mathematical foundation for broadcast-based and decentralized control strategies, where each agent uses only local information to achieve global flocking behavior.

Practical Applications of the Cucker–Smale Model

Beyond direct engineering control, the Cucker–Smale model is also important in modeling and analysis of complex collective systems.

- **Collective behavior in complex networks:** The model is used to study emergent phenomena in systems with many interacting agents, such as sensor networks, autonomous vehicle platoons, and distributed robotic systems with time-varying communication topologies.
- **Biological and social systems modeling:** It serves as a simplified but powerful framework for understanding flocking of birds, schooling of fish, crowd motion, and opinion dynamics in social systems.
- **Optimal control and kinetic models:** Kinetic and controlled versions of the Cucker–Smale model are used to design optimal strategies that steer a large population of agents toward consensus or flocking using minimal control effort.

Conclusion: The Cucker–Smale model provides a unifying mathematical framework for understanding and controlling collective dynamics in both natural and engineered systems.

Problem Formulation and Modeling Challenge

Problem. Given time-series observations

$$\mathcal{D} = \{(t_k, x_i(t_k), v_i(t_k))\}_{i=1,\dots,N}^{k=1,\dots,T},$$

our goal is to learn the underlying interaction dynamics **directly from data** while preserving **continuous-time structure**.

Modeling Challenge. Replacing the hand-crafted interaction term with a data-driven model that:

- operates in continuous time,
- captures nonlinear and unknown interactions,
- generalizes across time discretizations.

Proposed Approach. We model the dynamics using a **Neural Ordinary Differential Equation (Neural ODE)**, where the interaction term is parameterized by a neural network learned from data.

Problem Identification and Formulation

We consider a multi-agent dynamical system of **Cucker–Smale type**, where the motion of N interacting agents is governed by **unknown interaction forces**.

We are given time-series observations of agent positions and velocities:

$$\mathcal{D} = \left\{ \left(t_k, x_i(t_k), v_i(t_k) \right) \right\}_{k=1,\dots,T;\ i=1,\dots,N}.$$

- $t_k \in [0, T]$ are observation times,
- $x_i(t_k) \in \mathbb{R}^d$ denotes the position of agent i,
- $v_i(t_k) \in \mathbb{R}^d$ denotes the velocity of agent i.

The full system state at time t is

$$z(t) = \left(x_1(t), \dots, x_N(t), v_1(t), \dots, v_N(t) \right) \in \mathbb{R}^{2Nd}.$$

Nature of the data:

- high-dimensional state space ($\mathbb{R}^{2Nd}$),
- correlated trajectories (not i.i.d. samples),
- sampled from an underlying continuous-time dynamical system,
- data generated by Cucker–Smale dynamics with unknown interaction law.

Core Modeling Challenge

The classical **Cucker–Smale model** is governed by

$$\dot{x}_i(t) = v_i(t), \qquad \dot{v}_i(t) = \frac{\lambda}{N} \sum_{j=1}^{N} \phi(\|x_j(t) - x_i(t)\|) \big(v_j(t) - v_i(t)\big),$$

where the interaction function $\phi(\cdot)$ is **hand-crafted**.

Key challenges:

- **Unknown interaction dynamics:** the functional form of $\phi(\cdot)$ is not assumed to be known.
- **Nonlinearity and coupling:** each agent interacts nonlinearly with all other agents.
- **Continuous-time structure:** the learned model should be independent of any fixed time discretization.
- **Generalization across time grids:** evaluation at arbitrary time points.

Modeling Objective and Proposed Approach

Our objective is to learn the underlying interaction dynamics directly from data by approximating the unknown vector field governing the system:

$$\dot{z}(t) = f\big(z(t), t\big),$$

using only observed trajectories.

We adopt a **Neural Ordinary Differential Equation (Neural ODE)** formulation:

$$\dot{z}(t) = f_\theta\big(z(t), t\big), \qquad f_\theta : \mathbb{R}^{2Nd} \times \mathbb{R} \to \mathbb{R}^{2Nd},$$

where θ denotes trainable parameters.

Key properties:

- continuous-time modeling, independent of time discretization,
- data-driven replacement of hand-crafted interaction laws,
- solver-agnostic formulation (e.g. RK4, adaptive solvers),
- trajectory-level learning via the loss

$$\mathcal{L}(\theta) = \sum_{k=1}^{T} \big\| z_\theta(t_k) - z(t_k) \big\|^2.$$

Synthetic Data Generation (Ground-Truth Dynamics)

Cucker–Smale model (continuous time). For $i = 1, \dots, N$:

$$\dot{x}_i(t) = v_i(t), \qquad \dot{v}_i(t) = \frac{\lambda}{N} \sum_{j=1}^{N} \phi(\|x_j(t) - x_i(t)\|) \big(v_j(t) - v_i(t)\big),$$

with interaction kernel

$$\phi(r) = \frac{1}{(1 + r^2)^{\beta}}.$$

State-space formulation.

$$z(t) = \big(x_1(t), \dots, x_N(t), v_1(t), \dots, v_N(t)\big) \in \mathbb{R}^{2Nd}, \qquad \dot{z}(t) = f_{\mathrm{CS}}(z(t)).$$

Initial conditions.

$$x_i(0) \sim \mathcal{N}(0, 4I_d), \qquad v_i(0) \sim \mathcal{N}(0, I_d), \quad i = 1, \dots, N.$$

Data generation.

$$\text{Input: } z_0 \in \mathbb{R}^{2Nd}, \qquad \text{Output: } \{z(t)\}_{t \in [0,T]}.$$

Numerical Integration and Dataset Construction

Time grid. We consider the continuous time interval

$$t \in [0, T],$$

with a uniform observation grid

$$t_k = \frac{k}{K-1} T, \qquad k = 0, \ldots, K-1.$$

Experimental setting.

$$T = 10, \qquad K = 100.$$

Numerical solver. Ground-truth trajectories are obtained by solving

$$\dot{z}(t) = f_{\mathrm{CS}}(z(t)), \qquad z(0) = z_0,$$

using an adaptive Dormand–Prince Runge–Kutta method (DOPRI5):

$$z(t_k) = \mathrm{ODEINT}_{\mathrm{DOPRI5}}\big(f_{\mathrm{CS}}, z_0, t_k\big).$$

This provides an accurate continuous-time reference solution.

Generated dataset.

$$\mathcal{D} = \big\{(t_k, z(t_k))\big\}_{k=0}^{K-1}, \qquad z(t_k) \in \mathbb{R}^{2Nd}$$

Why Neural ODEs for Flocking Dynamics

Problem Setting

- Multi-agent flocking system (Cucker–Smale type)
- Governed by continuous-time ordinary differential equations
- Nonlinear and potentially unknown interaction mechanisms

Continuous-Time Modeling

- Neural ODEs parameterize the vector field:

$$\frac{dz(t)}{dt} = f_\theta(z(t), t)$$

- Preserves the intrinsic continuous-time structure
- Avoids discretization artifacts of fixed-step models
- Allows evaluation at arbitrary and unseen time points

Learning Latent Interactions and Generalization

Learning Latent Interaction Laws

- Classical flocking models assume a predefined interaction kernel
- In practice, interaction laws may be unknown or heterogeneous
- Neural ODEs learn effective interaction dynamics directly from data
- Captures emergent behaviors such as alignment and flock formation

Distribution-Free and Time-Invariant Modeling

- No assumption on data distribution or noise structure
- Dynamics learned directly from trajectory data
- Robust to irregular sampling and changing time resolutions
- Superior generalization compared to discrete-time sequence models

Why Alternative Models Are Less Suitable

- **Variational Autoencoders (VAEs):** Designed for latent distribution modeling; rely on discrete-time transitions and strong latent assumptions.
- **GANs:** Lack explicit temporal dynamics and do not define a state evolution law.
- **Normalizing Flows:** Focus on density estimation of static data, not continuous-time system evolution.
- **Diffusion Models:** Stochastic, discrete, and distribution-centric; unsuitable for deterministic physical dynamics.
- **Flow Matching:** Learn transport between distributions rather than governing differential equations.

These models prioritize data generation or likelihood modeling, whereas our task requires **learning continuous-time dynamics**.

Model Assumptions and Final Justification

Neural ODEs assume:

- Deterministic and smooth underlying dynamics
- Markovian state evolution
- Observations approximate the true continuous trajectory

Why this is appropriate:

- Classical flocking models satisfy all these assumptions
- The goal is system identification, not distribution generation
- Neural ODEs provide interpretability and physical consistency

Neural ODEs are the most principled choice for learning flocking dynamics.

Numerical Solver: Runge-Kutta (RK4)

Role in Neural ODE Pipeline:

- The Neural ODE framework defines continuous-time dynamics: $\frac{dz(t)}{dt} = f_\theta(z(t), t)$.
- To generate trajectories or perform predictions, the vector field f_θ must be integrated over time.
- RK4 (4th-order Runge-Kutta) is used as a fixed-step numerical integrator.

How RK4 Works:

- Estimates $z(t + h)$ using a weighted average of slopes:

$$k_1 = f_\theta(z_t, t), \quad k_2 = f_\theta(z_t + \frac{h}{2}k_1, t + \frac{h}{2}),$$

$$k_3 = f_\theta(z_t + \frac{h}{2}k_2, t + \frac{h}{2}), \quad k_4 = f_\theta(z_t + hk_3, t + h)$$

$$z_{t+h} = z_t + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

- Provides high-accuracy approximation for continuous trajectories.

Observed Dynamical System (Ground Truth)

Mathematical formulation

We observe trajectories of a multi-agent system governed by an unknown ordinary differential equation:

$$\frac{dz(t)}{dt} = F\big(z(t)\big), \qquad z(0) = z_0,$$

where the state vector is given by

$$z(t) = \big(x_1(t), \ldots, x_N(t), v_1(t), \ldots, v_N(t)\big) \in \mathbb{R}^{2Nd}.$$

Here:

- $x_i(t)$ denotes the position of agent i,
- $v_i(t)$ denotes the velocity of agent i.

Observed data

We are given discrete-time observations of the system:

$$\{z(t_k)\}_{k=0}^{T},$$

Learning Objective and Neural ODE Formulation

Goal

Approximate the unknown vector field F using a neural network:

$$F(z) \approx f_\theta(z, t),$$

such that the induced dynamical system reproduces the observed trajectories.

Neural ODE formulation

The learned dynamics are defined by the neural ordinary differential equation:

$$\frac{dz(t)}{dt} = f_\theta\big(z(t), t\big),$$

where the neural network satisfies

$$f_\theta : \mathbb{R}^{2Nd+1} \to \mathbb{R}^{2Nd}.$$

Time-Augmented State (Non-autonomous System)

Input augmentation

The neural network input is augmented with time:

$$\tilde{z}(t) = \big[\, z(t),\ t \,\big].$$

This allows modeling explicitly time-dependent dynamics:

$$\frac{dz}{dt} = f_\theta(z, t).$$

Implementation detail

In the code, this corresponds to concatenating the state with time:

```
z_t = torch.cat([z, t], dim=-1).
```

This enables the Neural ODE to capture time-dependent interactions.

Definition of the Vector Field f_θ

Neural network parameterization

The unknown vector field is approximated by a fully connected feedforward neural network:

$$f_\theta : \mathbb{R}^{2Nd+1} \to \mathbb{R}^{2Nd}.$$

Given the time-augmented input $[z, t]$, the network is defined as:

$$f_\theta(z, t) = W_3 h_2 + b_3,$$

with hidden layers

$$h_1 = \tanh(W_1[z, t] + b_1), \qquad h_2 = \tanh(W_2 h_1 + b_2).$$

Parameter dimensions

$$z \in \mathbb{R}^{2Nd}, \quad t \in \mathbb{R},$$
$$W_1 \in \mathbb{R}^{H\times(2Nd+1)}, \quad b_1 \in \mathbb{R}^H,$$
$$W_2 \in \mathbb{R}^{H\times H}, \quad b_2 \in \mathbb{R}^H,$$
$$W_3 \in \mathbb{R}^{2Nd\times H}, \quad b_3 \in \mathbb{R}^{2Nd}.$$

$$\theta = \{W_1, b_1, W_2, b_2, W_3, b_3\}.$$

Why tanh Activation (Not ReLU)

Activation function

The activation function is chosen as:

$$\sigma(x) = \tanh(x).$$

Rationale

- tanh is smooth and continuously differentiable.
- Neural ODEs require the vector field f_θ to be Lipschitz continuous.
- This ensures existence and uniqueness of ODE solutions.
- Avoids non-smooth vector fields induced by ReLU activations.

Smooth activations are crucial because the neural network defines a *differential equation*, not just a static mapping.

Forward Pass and Trajectory Loss

Trajectory generation

Given an initial condition z_0, the predicted trajectory is obtained by solving:

$$\hat{z}(t) = \text{ODESolve}\left(\frac{dz}{dt} = f_\theta(z,t),\ z(0) = z_0\right).$$

Discretely:

$$\hat{z}(t_k) = z_0 + \int_0^{t_k} f_\theta(z(t),t)\,dt.$$

Trajectory matching loss

The training objective compares predicted and observed trajectories:

$$L(\theta) = \frac{1}{T}\sum_{k=1}^{T} \left\|\hat{z}(t_k) - z(t_k)\right\|^2.$$

This functional loss:

- penalizes long-term divergence,
- enforces dynamical consistency,
- encourages learning the correct flow.

Why Backpropagation Is Non-trivial

Continuous optimization problem

We solve:

$$\theta^* = \arg\min_{\theta} L(\theta),$$

where the loss depends on θ through the solution of an ODE:

$$L(\theta) = L\big(\text{ODESolve}(f_\theta)\big).$$

Unlike standard neural networks:

$$L(\theta) \neq L\big(f_\theta(z)\big),$$

so gradients must pass through the ODE solver.

Adjoint sensitivity method

Define the adjoint variable:

$$\lambda(t) = \frac{\partial L}{\partial z(t)}.$$

It satisfies the backward ODE:

$$\frac{d\lambda}{dt} = -\left(\frac{\partial f_\theta}{\partial z}\right)^\top \lambda, \qquad \lambda(T) = \frac{\partial L}{\partial z(T)}.$$

Gradient Computation and Training Algorithm

Gradients with respect to parameters

$$\frac{dL}{d\theta} = \int_0^T \lambda(t)^\top \frac{\partial f_\theta(z(t), t)}{\partial \theta}\, dt.$$

Parameter update

Using gradient descent (or Adam):

$$\theta_{n+1} = \theta_n - \eta \nabla_\theta L.$$

Neural ODE training algorithm

1. Initialize parameters θ.
2. Solve forward ODE to obtain $\hat{z}(t)$.
3. Compute trajectory loss $L(\theta)$.
4. Solve adjoint ODE backward in time.
5. Compute gradients $\nabla_\theta L$.
6. Update parameters θ.

Experiment: Neural ODE for Cucker–Smale Dynamics

Objective

- Learn continuous-time Cucker–Smale dynamics from synthetic trajectory data
- Validate methodological correctness of the Neural ODE approach

Dataset

- Synthetic trajectory generated from Cucker–Smale ODE
- Number of agents: $N = 10$, dimension: $d = 2$
- State dimension: 40
- Time horizon: $T = 10$, uniformly sampled at 100 time points

Model and Training

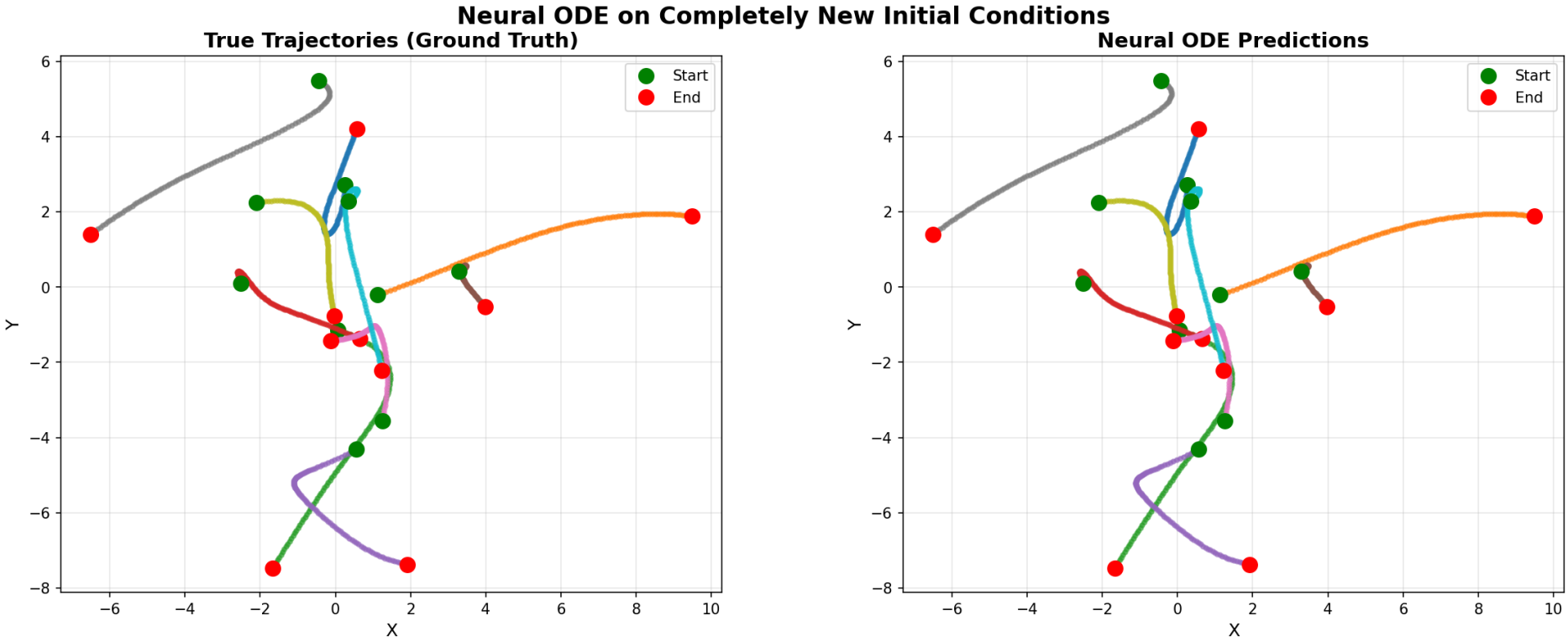
- Neural ODE with MLP vector field
- Architecture: $[41 \to 128 \to 128 \to 128 \to 40]$
- Activation function: tanh
- ODE solver: Runge–Kutta 4 (RK4)
- Loss function: trajectory-level mean squared error (MSE)

Training Output: Neural ODE with Early Stopping

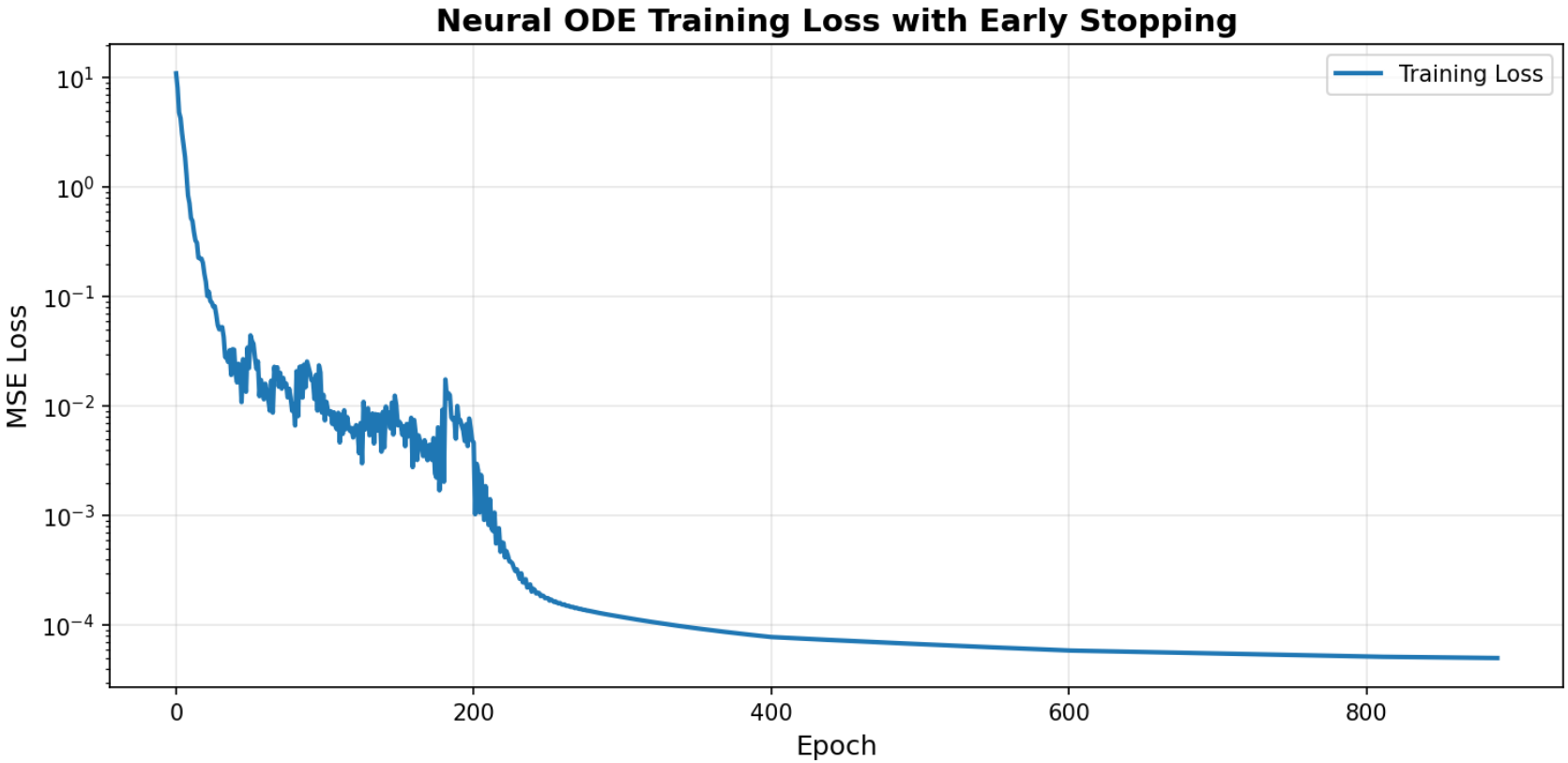
```
Early stopping triggered at epoch 850 Best loss: 0.000048 Training complete
at epoch 850 Final loss: 0.000047
```

Epoch	Loss	Best Loss
50	0.014712	0.011765
100	0.013810	0.005003
150	0.004547	0.002845
200	0.001722	0.001722
250	0.000153	0.000153
300	0.000111	0.000112
350	0.000088	0.000088
400	0.000073	0.000073
450	0.000067	0.000068
500	0.000063	0.000063
550	0.000058	0.000059
600	0.000055	0.000055
650	0.000053	0.000053
700	0.000051	0.000052
750	0.000050	0.000050
800	0.000048	0.000048

Neural ODE on Completely New Initial Condition

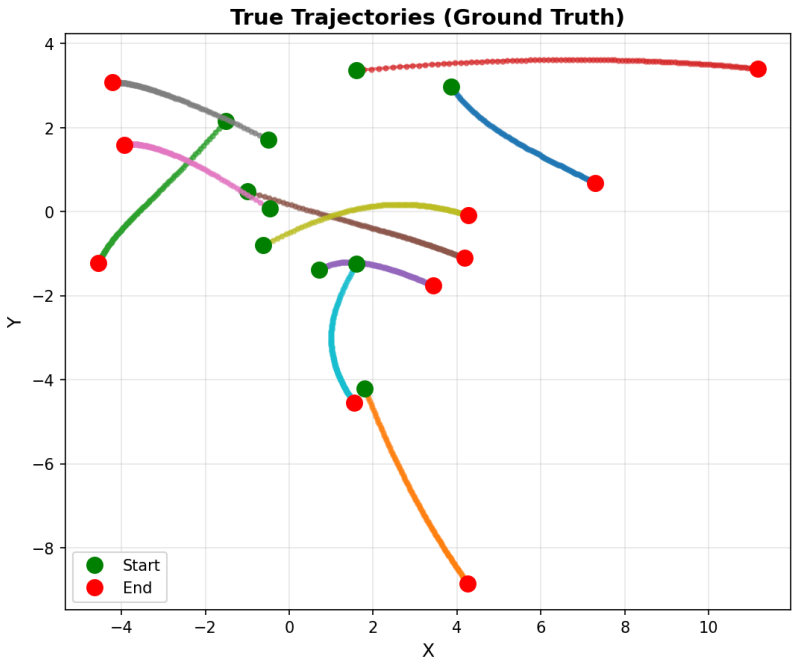


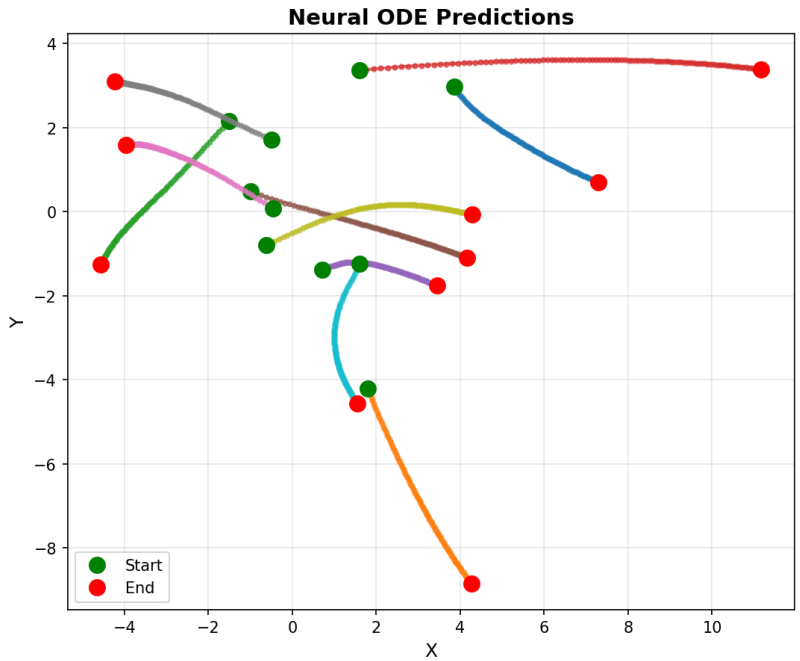
Training Loss with Early Stopping



Early stopping triggered at epoch 850 (patience = 50)

Final Trajectories Comparison





Experiment 2: Neural ODE for Cucker–Smale Dynamics

Objective:

- Learn continuous-time Cucker–Smale dynamics from synthetic trajectories
- Evaluate scalability: N=20, d=3
- Test generalization to new initial conditions

Dataset:

- Synthetic trajectories from Cucker–Smale ODE
- Agents: N=20, Dimension: d=3, State dim: 120
- Time horizon: T=15 s, sampled at 200 points

Model & Training:

- Neural ODE with MLP vector field
- Architecture: $[121 \to 256 \to 256 \to 256 \to 120]$, activation: tanh
- ODE solver: RK4
- Loss: trajectory MSE
- Optimizer: Adam, lr=5e-4; early stopping: patience=100, min_delta=1e-6
- Max epochs: 1500

Training Progress (Neural ODE with RK4)

Epoch	Loss	Best Loss
50	2.049099	2.049099
100	0.183677	0.182565
150	0.056535	0.041514
200	0.047467	0.032594
250	0.024472	0.012304
300	0.011748	0.010062
350	0.008210	0.007152
400	0.009593	0.006006
450	0.005231	0.004205
500	0.004293	0.004205
550	0.013093	0.003099
600	0.006793	0.003099
650	0.001989	0.001983
700	0.001981	0.001829
750	0.002763	0.001594
800	0.002479	0.001594
850	0.001358	0.001355
900	0.001267	0.001267
950	0.001190	0.001190
1000	0.001121	0.001121
1050	0.001087	0.001088
1100	0.001055	0.001056
1150	0.001024	0.001024

Neural ODE Training Analysis

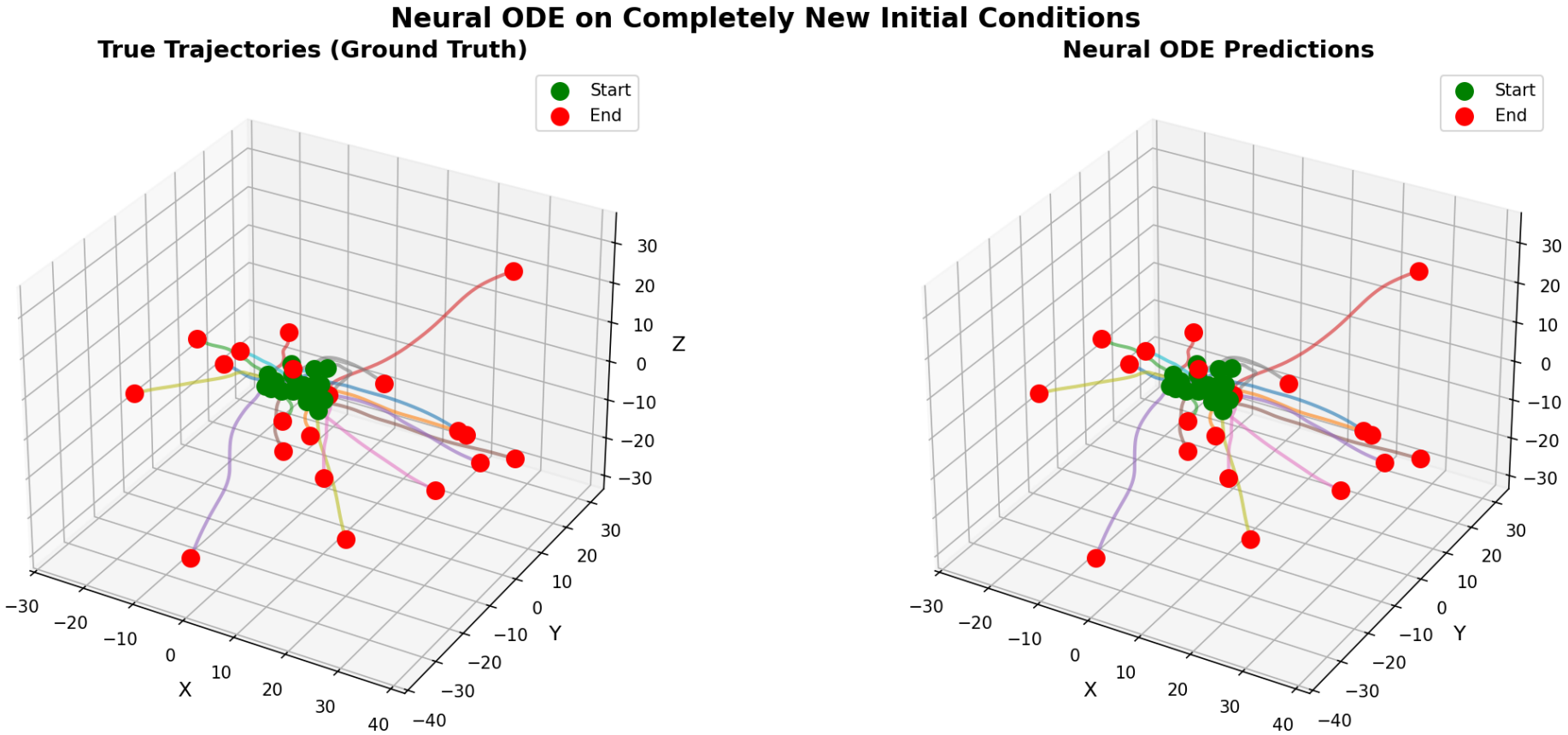
Observations from Training:

- Rapid initial decrease in loss: Epoch 50 $\rightarrow$ 2.05 to Epoch 150 $\rightarrow$ 0.056
- Gradual convergence after Epoch 300, reaching final loss 0.000917 at Epoch 1500
- Early stopping allowed model to stop before overfitting
- RK4 solver ensured stable and accurate integration of ODE dynamics
- Model successfully learned continuous-time dynamics of the system

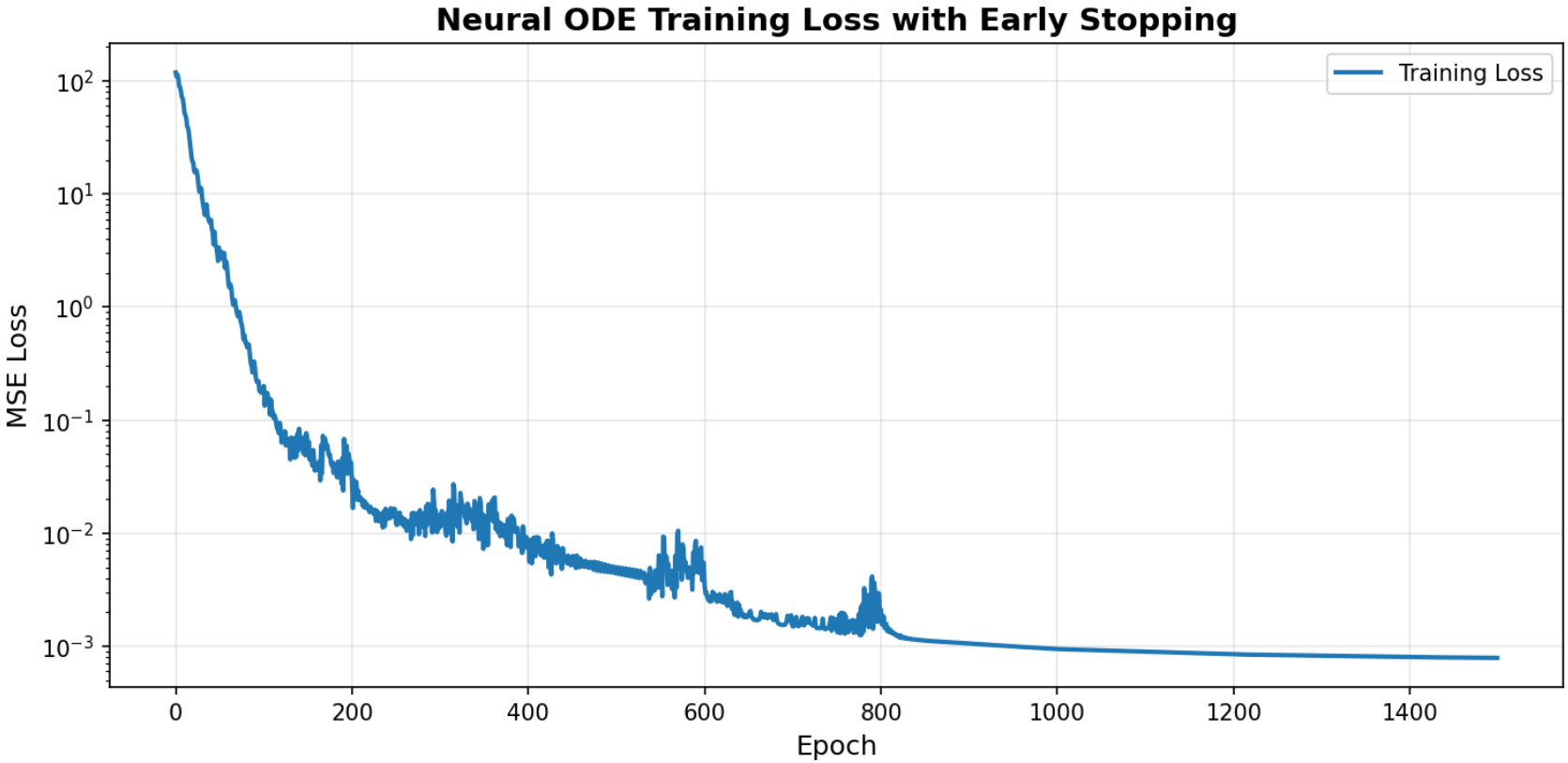
Training Trend:

Epoch	Loss	Trend
50	2.0491	High, initial
150	0.0565	Rapid decrease
300	0.0117	Slower decrease
650	0.001983	Converging
1500	0.000917	Final loss, well-trained

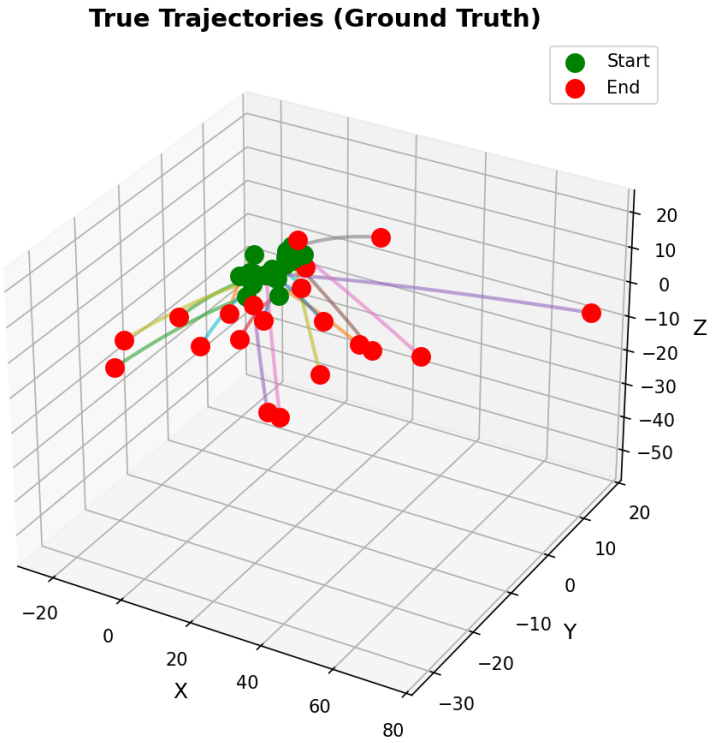
Neural ODE on Completely New Initial Condition

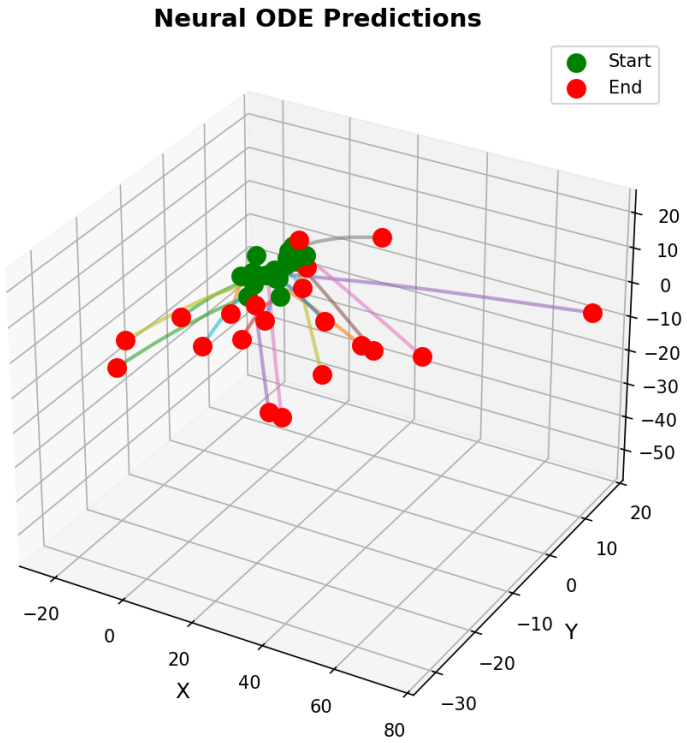


Training Loss with Early Stopping



Final Trajectories Comparison





Neural ODE Training Comparison

Experiment 1: N=5, RK4 solver
Final loss: 0.000917 at Epoch 1500
Experiment 2: N=10, RK4 solver
Final loss: 0.000047 at Epoch 850

Training Progress Comparison:

Epoch	Exp 2 Loss	Exp 2 Best	Exp 1 Loss	Exp 1 Best
50	2.0491	2.0491	0.0147	0.0118
850	0.001358	0.001355	0.000048	0.000048

Observations:

- Exp 1 (larger N=10) converges faster (early stopping at 850 epochs vs 1500)
- Exp 1 achieves lower final loss (0.000047 vs 0.000917)
- Both experiments show rapid initial decrease and slower convergence later
- RK4 solver ensures stability in both cases

References

- Ha & Liu (2009): A Simple Proof of the Cucker–Smale Flocking Dynamics and Mean-Field Limit
- Ren et al. (2025): Pattern Formation of Cucker–Smale System with Nonlinear Velocity Couplings
- Erban et al. (2016): A Cucker–Smale Model with Noise and Delay
- Axelsson (2022): Neural ODEs — Uppsala University Project Report

Thank you for your attention!